`gjxl`: A GPU-Accelerated JPEG XL Encoder

Yunho Cho and Sangeon Jeon

Georgia Institute of Technology
Atlanta, Georgia,
United States
yunhoyunho@gatech.edu
sjeon84@gatech.edu

Abstract

JPEG XL offers state-of-the-art compression rate and versatility. However, this comes at the cost of high computational complexity, especially for encoding. High-resolution images, such as those shot on modern smartphones, can take over 10 seconds to encode at high effort, which may complicate widespread adoption of JPEG XL. Based on the observation that much of `libjxl`'s expensive computations are parallelizable, we empirically demonstrate the usefulness of GPUs for JPEG XL by developing `gjxl`, a GPU-accelerated encoder that offloads perceptual analysis workloads to the GPU while keeping entropy/bitstream workloads on the CPU. `gjxl` achieves 2.8× speedup vs. `libjxl` at default effort while simultaneously producing smaller files via a more aggressive use of compute-intensive features. `gjxl` is available at https://github.com/yunho-c/gjxl.

Introduction

The continued growth of imaging and computing technologies suggests that vast amounts of digital storage will remain occupied by images and videos, underscoring the importance of efficient codecs. We have also witnessed the proliferation of machine learning in recent years, catalyzed by the use of GPUs for parallelizable computation in neural network training and inference [1]. In appropriate scenarios, GPUs offer higher FLOPs per given power and die area than CPUs, and have thus become first-class targets even for non-graphical compute workloads such as rigid-body physics simulation [2], signal processing [3], and linear algebra [4].

JPEG XL [5] is a modern image codec that achieves strong compression rates thanks to an intricate encoder design spanning perceptual analysis, image filtering, adaptive quantization, and context-based prediction. However, its adoption has been stagnant—partly due to performance implications, including slow encoder/decoder speeds and energy consumption for portable devices.

Foundational computations in image compression such as color transform, discrete cosine transform (DCT), and quantization expose data parallelism, creating a natural opportunity for GPU acceleration, as discussed by [6], [7]. On the other hand, the complexity of JPEG XL's encoder—which consists of a sequence of coupled encoder decisions involving search, sampling, and iteration—makes it nontrivial to efficiently utilize numerical accelerators, which prefer predictable memory access and simple control flow.

To investigate GPU acceleration for JPEG XL, we pose three research questions:

1. How is the runtime of `libjxl` distributed within encoding stages?

2. What kinds of parallelism is available in the underlying computations?
3. Can we achieve GPU-based speedup while retaining `libjxl`'s overall encoder design/heuristics, solely from equivalent parallel reformulations?

Based on our findings, we present `gjxl`—a JPEG XL encoder that exploits data/task-parallelism and reorganizes perceptual analysis workloads around a GPU-resident pipeline—that attains 1.55× to 36.9× encoding throughput vs. `libjxl` at matched effort.

Background

JPEG XL is an open, royalty-free image codec standardized under ISO/IEC 18181 with two operating modes: Variable DCT (*VarDCT*) and *Modular*. The former is designed for lossy encoding and natural/photographic images, whereas the latter is designed for lossless encoding and screen-space content. Both benefit from sophisticated late-stage entropy coding, utilizing asymmetric numeral system (ANS) [8] and context-aware prediction. JPEG XL provides `libjxl` [9] as the reference codec implementation.

VarDCT encoding. The VarDCT mode involves similar overarching operations to the original JPEG format, with the addition of perceptual analysis and adaptive allocation mechanisms. For instance, it uses adaptively sized (DCT) transform blocks unlike JPEG's uniform 8x8 blocks and content-dependent quantization strengths instead of a global value. The VarDCT encoding contains the following stages: 1) color transform, 2) perceptual analysis, 3) chroma-from-luma (CfL) decorrelation, 4) AC strategy selection, 5) adaptive quantization, 6) entropy coding, and 7) bitstream generation.

Related works. GPU acceleration has been investigated extensively for earlier image coding standards, showing that regular transform stages that expose fine-grained data parallelism map naturally to massively parallel hardware, whereas entropy coding, context modeling, and bitstream construction require careful reformulations due to data dependencies, irregular memory access, and control-flow divergence [6]. Works on JPEG entropy coding have used parallel scans and staged bitstream assembly to address the difficulty of producing variable-length output in parallel [10]. Subsequent work explored aggressive codec redesigns, such as reformulating JPEG 2000's bitplane coding to expose fine-grained SIMD parallelism [11] or developing a GPU-oriented codec that minimizes intermediate memory transfers and combines operations into a small number of kernels [12]. `gjxl` applies GPU acceleration to the substantially more complex JPEG XL encoder, focusing on parallelization and GPU-oriented reorganization, without invasive modification to libjxl's encoder heuristics.

Runtime characterization of `libjxl`

We profile VarDCT encoding in `libjxl` to analyze encoding speed with respect to effort, and to identify operations that dominate total encoding latency.[1] Table 1 shows that the large stepwise slowdowns are driven by the enablement of effort-gated features. At lower efforts, `libjxl` uses comparatively simple transform and quantization decisions, leaving

[1] Evaluated for CLIC'25 (2.8 MP/image) at quality 80 (distance 1.9). Measurements use a Release build on Apple M4 Pro with eight threads.

coefficient coding and entropy processing as the main cost. At medium efforts (5-7), AC strategy search accounts for around 50% of the total runtime, with AR filter selection responsible for an additional 25% for efforts 6-7. At high efforts (8-10), perceptual refinement accounts for over 85% of the total runtime.

Table 1: Runtime contribution of `libjxl` encoding stages at varying encoder efforts.

Effort	AC search	AR filter selection	Perceptual refinement	Tokenization & bitstream	Other	Total
1	0.1 ms	0 ms	0 ms	**11.2 ms**	6.9 ms	18.2 ms
2	0.1 ms	0 ms	0 ms	**11.3 ms**	7.3 ms	18.8 ms
3	0.1 ms	0 ms	0 ms	**12.5 ms**	7.1 ms	19.8 ms
4	0.1 ms	0 ms	0 ms	**15.7 ms**	7.1 ms	22.9 ms
5	**34.6 ms**	0 ms	0 ms	22.3 ms	9.7 ms	66.7 ms
6	**48.7 ms**	33.6 ms	0 ms	22.0 ms	11.7 ms	116.0 ms
7	**58.1 ms**	32.6 ms	0 ms	21.5 ms	16.3 ms	128.6 ms
8	57.7 ms	37.0 ms	**1373.5 ms**	62.9 ms	60.7 ms	1591.9 ms
9	57.9 ms	39.9 ms	**2179.8 ms**	148.9 ms	62.0 ms	2488.4 ms
10	59.7 ms	35.5 ms	**2113.7 ms**	146.1 ms	67.6 ms	2422.6 ms

For each effort, the longest stage is boldfaced and the second is underlined.

We consider these runtime-dominating operations as the primary optimization target for GPU acceleration in `gjxl`, and briefly describe their computation and parallelism below.

AC strategy search is responsible for a critical functionality in JPEG XL: partitioning of the image into variable-sized DCT blocks (*AC candidate*) to optimize the rate-distortion tradeoff, assigning finer blocks to regions with high visual complexity and vice versa. This involves two forms of computations, candidate evaluation and partition construction, which have different dependencies. Costs of multiple candidates can be evaluated in parallel, since each candidate's cost depends only on the source image, initial quantization field, masking information, and color-correlation parameters. Partition construction is a stateful process that must resolve overlaps and preserve the search's merge priorities within regularly sized tiles. In practice, candidate evaluation uses a cheap rate-distortion proxy $C_{a,s}$ expressed as (1) for candidate anchor a and shape s, where m_s is a shape-dependent rate multiplier, d is the requested perceptual target, N_s is the number of coefficients per channel, and $\bar{q}_{a,s}$ aggregates the initial quantization field over the candidate footprints. The rate proxy $\widehat{R}_{a,s}$ combines quantized-coefficient magnitudes and nonzero-count terms. The loss term $L_{a,s}$ is a masked, channel-weighted eight-power sum of inverse-transformed rounding residuals.

$$C_{a,s} = m_s \widehat{R}_{a,s} + \lambda(d) \frac{N_s}{\bar{q}_{a,s}} \left(\frac{L_{a,s}}{N_s} \right)^{1/8} \tag{1}$$

`libjxl` reduces the combinatorial search space of the partition construction step by performing dense evaluation of local ($\leq$8x8) strategies, followed by a hierarchical greedy merge heuristic that evaluates progressively larger ($\leq$64x64) strategies.

Perceptual refinement involves comparing the frequency-transformed and quantized candidate image with the original image to identify pixelwise perceptual errors, and updating the quantization weight field to iteratively adjust precision towards target quality.

This uses the Butteraugli full-reference perceptual error metric [13], where images are transformed into an opponent-color representation related to the retinal response, after which differences are evaluated across multiple spatial-frequency scales. The metric applies perceptual masking, accounting for the reduced visibility of distortions in regions containing sufficiently strong image structure or activity—modeling differences in human sensitivity to luminance, chromatic variation, spatial frequency, and local image content.
We define $E_t(b)$ as the aggregate perceptual error for AC strategy block b at refinement iteration t and compute using (2). Here, P_b represents source pixels, and $\delta_t(p)$ is the Butteraugli distance at pixel p. This high-order norm gives substantial weight to localized errors.

$$E_t(b) = 1.2 \left[\frac{1}{|P_b|} \sum_{p \in P_b} \delta_t(p)^{16} \right]^{1/16} \tag{2}$$

The update policy compares $E_t(b)$ with the target perceptual distance d. Above-target error increases the quantization field in proportion to $E_t(b)/d$; below-target error can decrease the field by the factor $E_t(b)/d^{0.2}$, recovering compression headroom.

Tokenization. Between final quantization and entropy coding, the encoder converts transform coefficients into a sequence of (unsigned integer, entropy context ID) pairs or *tokens*. DC coefficients (i.e., average block color) are coded separately from AC coefficients, due to different statistical properties e.g., stronger spatial autocorrelation. In DC tokenization, for each 2048x2048 region, the encoder predicts the DC value of each 8x8 block from neighboring values and converts the prediction error into an integer token, paired with an entropy context. In AC tokenization, for each block and color channel, the encoder determines and tokenizes the number of nonzero AC coefficients r_k, which enables an end-of-block signaling mechanism and provides a running sparsity estimate used for entropy-context selection. The coefficients are then traversed according to an adaptive scan order—where each visited coefficient is mapped to an unsigned integer and assigned an entropy context based on its block characteristics, scan position, remaining nonzero count, and preceding-coefficient state. The tokens are subsequently entropy-coded into the final bitstream. Zero-testing and signed-to-unsigned mapping are independent across coefficients, while remaining nonzero counts form a prefix-sum computation. Thus, coefficient-level parallelism can be exposed through reductions and parallel scans despite the traversal algorithm maintaining sequential scan states.

Methods

System design. `gjxl` is a JPEG XL VarDCT encoder implemented in C++ with CUDA and Metal GPU backends that accelerates the encoding pipeline using a combination of dataflow and kernel optimizations. `gjxl` is heavily inspired by, and builds on, JPEG XL coding tools and `libjxl`'s encoder design/heuristics. Our design choices are as follows:

- We load and store data (e.g., images, quantization fields, AC strategy maps, transformed coefficients, and reconstructions) directly on GPU memory to minimize host-device data transfer.
- We place the GPU call boundary around the entire early-stage pipeline (AC search, adaptive quantization, perceptual refinement, and tokenization) rather than individually offloading computations, to amortize synchronization latency.

- For suitable computations, we exploit data parallelism and further optimize individual kernels using kernel fusion, data reuse in register files and shared/threadgroup memory, and tiling. We discuss kernel-level optimizations for *AC strategy search*, *perceptual refinement*, and *AC tokenization* in detail below.

Figure 1 describes the high-level execution flow of an encoding operation. The GPU performs input preparation, AC strategy search, iterative perceptual refinement, and AC tokenization, producing quantized coefficients and metadata tokens, which are entropy-coded on the CPU to generate the final JPEG XL bitstream. Key intermediate data reside in GPU memory to minimize host-device data movement. Tokenization uses CPU-GPU overlap to exploit task parallelism between DC and AC coefficients, hiding latency from serial-heavy DC tokenization stage and kickstarting DC entropy coding, which involves a *low-frequency substream* independent of the full image in JPEG XL.

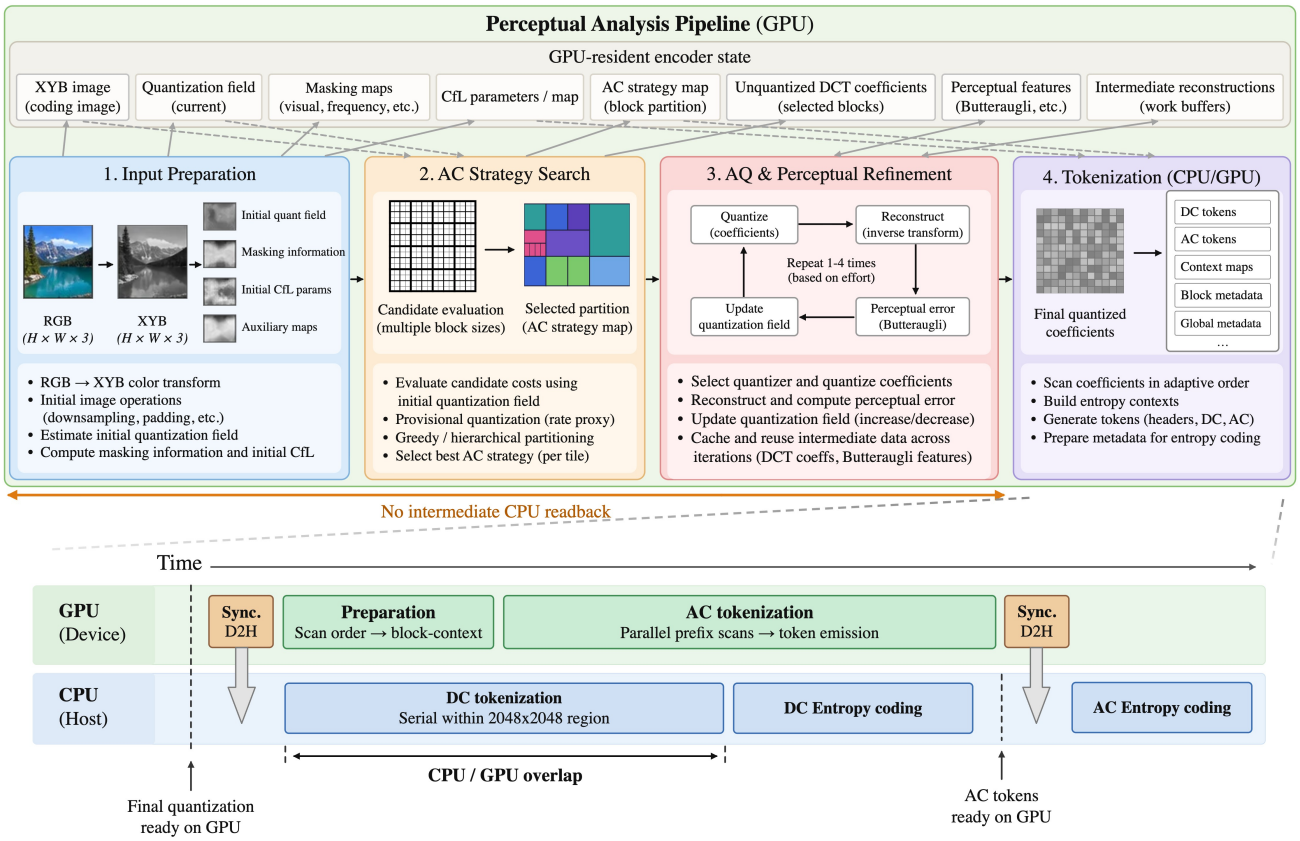


Figure 1: Overview of encoding pipeline in `gjxl`.

AC strategy search. During candidate evaluation, we batch candidates by transform shape and assign one block/threadgroup to each candidate. Threads within the group cooperate across the three color channels to respect Y→X,B chroma-from-luma dependency. The forward and residual inverse transforms use separable DCTs expressed as radix-2 decompositions/tiled 8x8 matrix products, evaluated with ordinary CUDA/SIMD-group matrix operations. The fused kernel performs pixel gathering, forward transform, chroma decorrelation and provisional quantization, residual inverse transformation, and spatial-loss reduction.

Each threadgroup gathers its candidate's pixels directly from the resident coding image. DCT basis matrices are staged once into shared/threadgroup memory and shared across channels and between the forward and inverse transforms. Intermediate data remain in shared/threadgroup memory and are overwritten/repurposed after final use. For instance, pixel storage becomes proxy cost reduction scratch and then holds inverse-transform output. The fused kernel writes per-channel rate and loss statistics, which a separate finalization kernel combines into one candidate cost. Thus, the intermediate coefficient and spatial-residual arrays do not pass through device memory between these operations.

The partition construction kernel assigns one GPU thread to each 64x64-pixel tile. Each thread reads the candidate costs directly from the transform-family buffers and executes the greedy placement procedure. Selection advances independently across tiles, with each thread writing the strategy and anchor information for its own memory region. This mapping exposes parallelism across tiles, without inter-tile synchronization or conflicts.

Perceptual refinement. A single perceptual refinement pass consists of quantization, reconstruction, perceptual error evaluation, and update. We pack multiple passes into a single GPU command buffer—the updated quantization field immediately feeds the next. Notably, certain quantities do not depend on quantization weights and thus are computed once and retained across all evaluations: 1) CfL map, which is derived from the initial quantization field, 2) selected partition, which is held fixed after *AC strategy search*, 3) unquantized forward DCT coefficients, and 4) Butteraugli features of the original image.

The kernel first derives the global quantizer and integer block quantization values from the current field, then performs quantizer selection based on median and median absolute deviation (MAD) statistics obtained by radix selection. Quantized coefficients are computed from cached coefficients using the selected quantizer and then reconstructed through dequantization, inverse transform, and filtering.

We use kernel fusion to reduce data movement during reconstruction and perceptual evaluation. The final EPF pass fuses XYB→RGB color transform for Butteraugli ingestion. Within Butteraugli, fused Malta kernels compute scaled differences in a shared tile with halo pixels and evaluate neighborhood responses before accumulating their contribution. The kernels combine multiscale composition with transform-aware error reduction, avoiding in-memory materialization of unnecessary distance-map intermediates.

Tokenization. We exploit task parallelism by overlapping DC tokenization (CPU) and AC tokenization (GPU). AC tokenization starts by computing coefficient scan orders and block-context map from GPU-resident final quantized coefficients. Then, a metadata kernel assigns one 32-lane warp/SIMD group to each block-channel pair and computes its nonzero count and last nonzero position. These determine the stream length: a count token followed by all coefficients through the last nonzero, including intervening zeros. The kernel constructs a spatial map of size-normalized nonzero counts. Since neighboring-block prediction depends solely on this quantity rather than previous tokens, precomputing the map permits independent context evaluation during token emission.

Prefix sums within and across AC groups assign disjoint output intervals in coding order. During emission, a SIMD prefix sum of nonzero indicators reconstructs the number of nonzeros remaining at each coefficient, while lane shuffles recover the preceding coefficient's nonzero flag. These operations reproduce the CPU context calculation without serializing individual coefficients. Tokens occupy separate arrays of 32-bit values and 16-bit

contexts in reusable shared buffers, and completed SoA token arrays are consumed directly by the CPU.

Encoding effort. `gjxl` drastically reduces the cost of compute-intensive perceptual operations. We promote such stages so they are enabled earlier in the effort range (Table 2), to let users more intuitively select encoder effort.

Table 2: Comparison of effort ladder for VarDCT encoding.

Effort	**`libjxl`**	**`gjxl`** (differences in bold)
1, 2	Uniform block size (8x8 DCT), adaptive DC smoothing	Uniform block size (8x8 DCT), adaptive DC smoothing, **ANS entropy coding, adaptive coefficient ordering**
3	ANS entropy coding	
4	Adaptive coefficient ordering, prediction-aware DC quantization	Prediction-aware DC quantization
5	Simple AC strategy search, adaptive quantization, Gabor-like transform, chroma-from-luma	**Full** AC strategy search, adaptive quantization, Gabor-like transform, chroma-from-luma
6	Full AC strategy search	**1 AQ update**
7	Patch deduplication	**2 AQ updates**, patch deduplication
8	2 AQ updates, rate-optimized entropy search, nonlinear final CfL	**3 AQ updates**, rate-optimized entropy search, nonlinear final CfL, **more exhaustive histogram clustering and numeric representation search**
9	4 AQ updates, more exhaustive histogram clustering and numeric representation search	4 AQ updates
10	4 AQ updates, finer AC strategy search stride	4 AQ updates, finer AC strategy search stride

Results

Experimental setup. We evaluate release-build `gjxl` v0.0.1 and `libjxl` v0.12.0 on Machine A (Apple M4 Pro, 48 GB RAM, 20-core GPU) and Machine B (Ryzen R7 4800HS, 40 GB RAM, RTX 3060 Mobile) for the following datasets: 1) Kodak Lossless True Color Image Suite, 2) Challenge on Learned Image Compression (CLIC) 2025, and 3) a custom high-resolution (12-48MP) corpus sourced from Unsplash. Encode time is measured from image submission to encoded bitstream production in memory, excluding filesystem operations, using 8 CPU threads for all scenarios. We compare the relationship between encode time and compression rate by evaluating Bjøntegaard-Delta (BD) rate [14], which quantifies compression rate difference over a range of quality values. BD-rate is computed over a common SSIMULACRA2 interval from 5 sampled quality points using PCHIP.

Performance. `gjxl` achieves 2.8× speedup over `libjxl` at default effort, with the speedup ranging from 1.55× to 36.9× as a function of encoder effort (Table 3). Simultaneously, `gjxl` achieves 0.71-9.14% smaller encoded size at matching output quality when using the revised effort policy. Figure 2 shows that `gjxl` dominates the speed-compression Pareto frontier, achieving more compression for any given time budget within tested configurations.

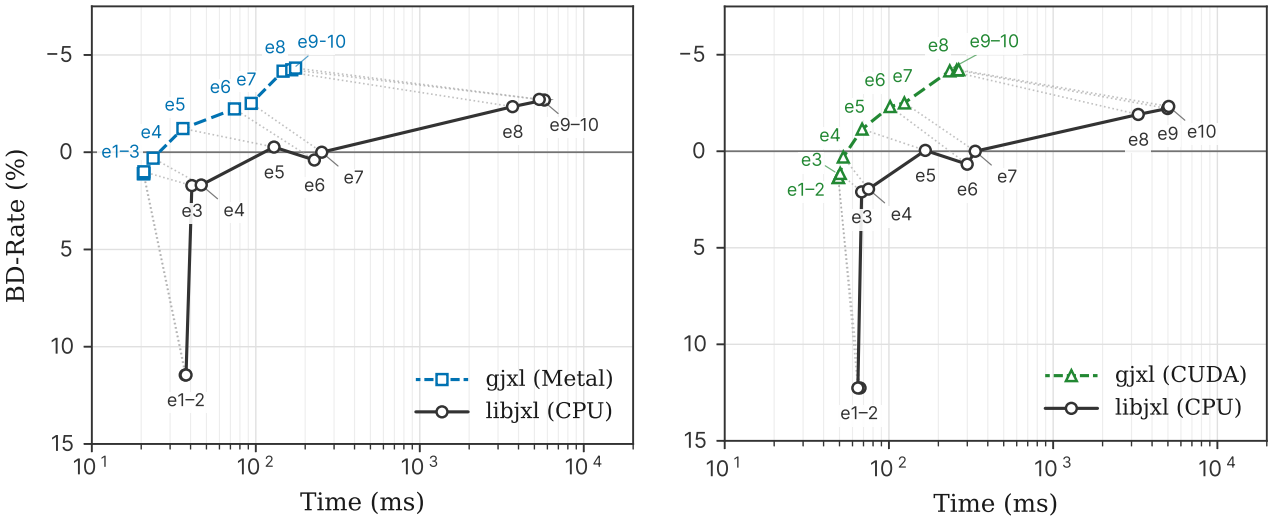


Figure 2: Encode time vs. BD-rate.[2]
(Left: Machine A, Right: Machine B)

Table 3: Encoding throughput and BD-rate difference.

Effort	Machine A (Metal)			Machine B (CUDA)			BD-rate Δ
	`libjxl`	`gjxl`	Speedup	`libjxl`	`gjxl`	Speedup	
1	149.5 MP/s	290.3 MP/s	1.94×	50.0 MP/s	77.5 MP/s	1.55×	-9.14%
2	147.7 MP/s	287.8 MP/s	1.95×	49.8 MP/s	78.6 MP/s	1.58×	-9.14%
3	138.3 MP/s	288.0 MP/s	2.08×	48.8 MP/s	77.9 MP/s	1.60×	-0.71%
4	121.2 MP/s	248.8 MP/s	2.05×	44.4 MP/s	73.0 MP/s	1.65×	-1.33%
5	43.3 MP/s	163.4 MP/s	3.78×	19.0 MP/s	49.0 MP/s	2.58×	-1.06%
6	25.7 MP/s	76.0 MP/s	2.96×	11.3 MP/s	32.2 MP/s	2.84×	-3.87%
7	20.8 MP/s	60.0 MP/s	2.88×	9.37 MP/s	25.7 MP/s	2.74×	-3.83%
8	1.46 MP/s	39.2 MP/s	26.86×	0.92 MP/s	14.8 MP/s	16.04×	-1.81%
9	0.94 MP/s	34.5 MP/s	36.89×	0.61 MP/s	13.0 MP/s	21.37×	-1.49%
10	1.00 MP/s	32.6 MP/s	32.45×	0.61 MP/s	12.6 MP/s	20.81×	-1.54%

[2]BD-rate is calculated relative to `libjxl` e7 in SSIMU2 60-85 interval. Encoder distances are pre-calibrated to produce identical decoded SSIMU2 scores. Average image size is 3.3 MP/image.

Conclusion

We develop `gjxl`, a JPEG XL-compatible VarDCT encoder with GPU acceleration, and jointly evaluate compressed size, image quality, and encoding time.

Significance. `gjxl` is the first GPU-accelerated encoder for JPEG XL and demonstrates substantial end-to-end speedup—without modifying the overall `libjxl` encoder design. We believe our results and parallel reformulations may provide useful insights for future optimization of `libjxl` (e.g., to utilize advanced SIMD instructions), and that analyzing the shifted bottlenecks may provide useful insights for future image codec designs.

Limitations. `gjxl` does not support Modular mode, which uses squeeze-based transforms for lossless encoding, leaving space for further work. Codestream computations including entropy coding and bitstream assembly remain on CPU, and inhibit dramatic speedups due to Amdahl's law. `gjxl` has a substantial memory footprint (spanning gigabytes for high-resolution images), imposing VRAM-based encodable image size constraints.

Future directions. Modern computing hardware is increasingly equipped with dedicated accelerators for neural network inference (NPUs), which provide exciting possibilities for offloading operations in the encoder, e.g., convolution-heavy Butteraugli evaluation, or replacing them with learned surrogate models. Such hybrid encoding pipelines may advance the speed-compression frontier or simplify encoder design.

References

[1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems 25 (NeurIPS)*, 2012, pp. 1097–1105.

[2] NVIDIA, "Isaac Sim," 2026, Version 6.0.1, https://github.com/isaac-sim/IsaacSim.

[3] D. Tolmachev, "VkFFT—a performant, cross-platform and open-source GPU FFT library," *IEEE Access*, vol. 11, pp. 12039–12058, 2023, doi: 10.1109/ACCESS.2023.3242240.

[4] J. Bradbury et al., "JAX: composable transformations of Python+NumPy programs," 2018, http://github.com/jax-ml/jax.

[5] J. Alakuijala et al., "JPEG XL next-generation image compression architecture and coding tools," in *Proc. SPIE, Applications of Digital Image Processing XLII*, 2019, vol. 11137, pp. 112–124, doi: 10.1117/12.2529237.

[6] J. Matela, P. Holub, M. Jirman, and M. Šrom, "GPU-specific reformulations of image compression algorithms," in *Proc. SPIE, Applications of Digital Image Processing XXXV*, 2012, vol. 8499, p. 849916, doi: 10.1117/12.929971.

[7] A. Weißenberger and B. Schmidt, "Accelerating JPEG decompression on GPUs," in *Proc. 28th IEEE Int. Conf. High Perform. Comput., Data, and Analytics (HiPC)*, 2021, pp. 121–130, doi: 10.1109/HiPC53243.2021.00026.

[8] J. Duda, K. Tahboub, N. J. Gadgil, and E. J. Delp, "The use of asymmetric numeral systems as an accurate replacement for Huffman coding," in *Proc. Picture Coding Symp. (PCS)*, Cairns, QLD, Australia, 2015, pp. 65–69, doi: 10.1109/PCS.2015.7170048.

[9] JPEG XL Project, "libjxl: JPEG XL reference implementation," GitHub, https://github.com/libjxl/libjxl.

[10] F. Zhu and H. Yan, "An efficient parallel entropy coding method for JPEG compression based on GPU," *J. Supercomput.*, vol. 78, no. 2, pp. 2681–2708, Feb. 2022, doi: 10.1007/s11227-021-03971-6.

[11] P. Enfedaque, F. Aulí-Llinàs, and J. C. Moure, "GPU implementation of bitplane coding with parallel coefficient processing for high performance image compression," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 8, pp. 2272–2284, Aug. 2017, doi: 10.1109/TPDS.2017.2657506.

[12] C. de Cea-Dominguez, J. C. Moure-Lopez, J. Bartrina-Rapesta, and F. Aulí-Llinàs, "GPU-oriented architecture for an end-to-end image/video codec based on JPEG2000," *IEEE Access*, vol. 8, pp. 68474–68487, 2020, doi: 10.1109/ACCESS.2020.2985859.

[13] J. Alakuijala, R. Obryk, O. Stoliarchuk, Z. Szabadka, L. Vandevenne, and J. Wassenberg, "Guetzli: Perceptually guided JPEG encoder," arXiv preprint arXiv:1703.04421, 2017.

[14] G. Bjontegaard, "Calculation of average PSNR differences between RD-curves," Document VCEG-M33, ITU-T Video Coding Experts Group, Austin, TX, USA, Apr. 2001.